```
"""
トレンドシグナル分析ツール — クラウド対応・モバイル最適化版
═══════════════════════════════════════════════════════
Streamlit Community Cloud で動作する本番対応版。
APScheduler の代わりに st.cache_data + time.sleep による
ポーリング方式で安定した自動更新を実現。

デプロイ先: https://streamlit.io/cloud
"""

import warnings, time
from datetime import datetime

warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import streamlit as st
import yfinance as yf
import ta

# ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
#   定数
# ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━

PERIODS      = ["1mo", "3mo", "6mo", "1y", "2y", "5y"]
INTERVALS    = ["1d", "1wk", "1mo"]
PERIOD_LBL   = {"1mo":"1ヶ月","3mo":"3ヶ月","6mo":"6ヶ月","1y":"1年","2y":"2年","5y"
INTERVAL_LBL = {"1d":"日足","1wk":"週足","1mo":"月足"}
UPDATE_OPTS  = {"5分":300, "10分":600, "15分":900, "30分":1800, "1時間":3600}

POPULAR = [
    ("7203.T","トヨタ"),   ("6758.T","ソニー"),   ("9984.T","SBG"),
    ("6861.T","キーエンス"),("8306.T","三菱UFJ"),  ("^N225","日経225"),
    ("AAPL","Apple"),      ("NVDA","NVIDIA"),       ("^GSPC","S&P500"),
    ("USDJPY=X","ドル円"),
]

C = {
    "bg":"#0d1117","panel":"#161b22","grid":"#21262d",
```

```
    "text":"#e6edf3","sub":"#8b949e",
    "buy":"#3fb950","sell":"#f85149","neutral":"#58a6ff",
    "sma25":"#ffa657","sma75":"#58a6ff","sma200":"#bc8cff",
    "bb":"#388bfd","macd":"#58a6ff","msig":"#ffa657",
    "hup":"#3fb950","hdn":"#f85149","rsi":"#d2a8ff",
    "cup":"#3fb950","cdn":"#f85149",
}

# ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
#  分析エンジン（60秒キャッシュ）
# ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━

@st.cache_data(ttl=60, show_spinner=False)
def fetch_and_analyze(code: str, period: str, interval: str):
    try:
        df = yf.download(code, period=period, interval=interval,
                         auto_adjust=True, progress=False)
        if df is None or df.empty or len(df) < 30:
            return None
        df = df.dropna(subset=["Close","Open","High","Low","Volume"])

        close = df["Close"].squeeze()
        high  = df["High"].squeeze()
        low   = df["Low"].squeeze()
        vol   = df["Volume"].squeeze()

        df["SMA25"]  = close.rolling(25).mean()
        df["SMA75"]  = close.rolling(75).mean()
        df["SMA200"] = close.rolling(200).mean()

        bb = ta.volatility.BollingerBands(close, 20, 2)
        df["BB_upper"] = bb.bollinger_hband()
        df["BB_mid"]   = bb.bollinger_mavg()
        df["BB_lower"] = bb.bollinger_lband()
        df["BB_width"] = (df["BB_upper"] - df["BB_lower"]) / df["BB_mid"]

        m = ta.trend.MACD(close, 26, 12, 9)
        df["MACD"]      = m.macd()
        df["MACD_sig"]  = m.macd_signal()
        df["MACD_hist"] = m.macd_diff()

        df["RSI"]     = ta.momentum.RSIIndicator(close, 14).rsi()
        df["ATR"]     = ta.volatility.AverageTrueRange(high, low, close, 14).aver
        adx           = ta.trend.ADXIndicator(high, low, close, 14)
        df["ADX"]     = adx.adx()
        st_obj        = ta.momentum.StochasticOscillator(high, low, close, 14, 3)
        df["Stoch_K"] = st_obj.stoch()
```

```
        df["Stoch_D"] = st_obj.stoch_signal()
        df["Vol_MA20"]= vol.rolling(20).mean()

        s = df
        mxu = (s["MACD"]>s["MACD_sig"])&(s["MACD"].shift(1)<=s["MACD_sig"].shift(
        mxd = (s["MACD"]<s["MACD_sig"])&(s["MACD"].shift(1)>=s["MACD_sig"].shift(
        sxu = (s["Stoch_K"]>s["Stoch_D"])&(s["Stoch_K"].shift(1)<=s["Stoch_D"].sh
        sxd = (s["Stoch_K"]<s["Stoch_D"])&(s["Stoch_K"].shift(1)>=s["Stoch_D"].sh

        buy_sc = (
            (s["SMA25"]>s["SMA75"]).astype(int) + mxu.astype(int)*2 +
            ((s["RSI"]>30)&(s["RSI"].shift(1)<=30)).astype(int) +
            (close<=s["BB_lower"]*1.01).astype(int) +
            (s["ADX"]>25).astype(int) + (sxu&(s["Stoch_K"]<25)).astype(int)
        )
        sell_sc = (
            (s["SMA25"]<s["SMA75"]).astype(int) + mxd.astype(int)*2 +
            ((s["RSI"]<70)&(s["RSI"].shift(1)>=70)).astype(int) +
            (close>=s["BB_upper"]*0.99).astype(int) +
            (s["ADX"]>25).astype(int) + (sxd&(s["Stoch_K"]>75)).astype(int)
        )

        df["buy_sc"]  = buy_sc
        df["sell_sc"] = sell_sc
        df["signal"]  = 0
        df.loc[buy_sc  >= 3, "signal"] =  1
        df.loc[sell_sc >= 3, "signal"] = -1

        df["trend"] = "横ばい"
        df.loc[(s["SMA25"]>s["SMA75"])&(s["SMA75"]>s["SMA200"])&(s["ADX"]>20),"tr
        df.loc[(s["SMA25"]<s["SMA75"])&(s["SMA75"]<s["SMA200"])&(s["ADX"]>20),"tr

        return {"df": df, "fetched_at": datetime.now().strftime("%Y/%m/%d %H:%M:%
    except Exception as e:
        return {"error": str(e)}

def sig_label(v):
    return "▲ 買い" if v==1 else ("▼ 売り" if v==-1 else "— 中立")

# ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
#  チャート描画
# ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━

def draw_candles(ax, df):
    op = df["Open"].squeeze().values
    hi = df["High"].squeeze().values
    lo = df["Low"].squeeze().values
```

```
    cl = df["Close"].squeeze().values
    for i in range(len(df)):
        col = C["cup"] if cl[i]>=op[i] else C["cdn"]
        ax.plot([i,i],[lo[i],hi[i]], color=col, lw=0.7, zorder=2)
        b0, b1 = min(op[i],cl[i]), max(op[i],cl[i])
        ax.bar(i, max(b1-b0,1e-6), bottom=b0, width=0.6, color=col, linewidth=0,

def make_chart(df, title, mobile=False):
    w, h = (9,14) if mobile else (16,13)
    fig = plt.figure(figsize=(w,h), facecolor=C["bg"])
    fig.suptitle(title, fontsize=9 if mobile else 12, color=C["text"],
                 fontweight="bold", x=0.02, ha="left", y=0.99)
    gs = gridspec.GridSpec(5,1,figure=fig, height_ratios=[4,1,1.3,1.3,1.3],
                           hspace=0.05, top=0.96, bottom=0.04,
                           left=0.10 if mobile else 0.07, right=0.97)
    axes = [fig.add_subplot(gs[i]) for i in range(5)]
    for ax in axes:
        ax.set_facecolor(C["panel"])
        ax.tick_params(colors=C["sub"], labelsize=6 if mobile else 7.5)
        for sp in ax.spines.values(): sp.set_edgecolor(C["grid"])
        ax.grid(axis="y", color=C["grid"], lw=0.5, ls="--", alpha=0.7)
        ax.grid(axis="x", color=C["grid"], lw=0.3, alpha=0.4)

    x  = np.arange(len(df))
    ds = df.index.strftime("%m/%d").tolist()
    step = max(1, len(df)//(6 if mobile else 10))
    tks  = list(range(0,len(df),step))
    for ax in axes:
        ax.set_xlim(-1,len(df)); ax.set_xticks(tks)
    for ax in axes[:-1]: ax.set_xticklabels([])
    axes[-1].set_xticklabels([ds[i] for i in tks], rotation=35, ha="right",
                             fontsize=5 if mobile else 6.5)

    cv = df["Close"].squeeze().values

    ax1 = axes[0]
    draw_candles(ax1, df)
    ax1.plot(x, df["SMA25"].values,  color=C["sma25"],  lw=1.0, label="SMA25")
    ax1.plot(x, df["SMA75"].values,  color=C["sma75"],  lw=1.2, label="SMA75")
    ax1.plot(x, df["SMA200"].values, color=C["sma200"], lw=1.3, ls="--", label="S
    ax1.fill_between(x, df["BB_upper"].values, df["BB_lower"].values, alpha=0.1,
    ax1.plot(x, df["BB_upper"].values, color=C["bb"], lw=0.7, ls=":", alpha=0.8)
    ax1.plot(x, df["BB_lower"].values, color=C["bb"], lw=0.7, ls=":", alpha=0.8)

    bx = [df.index.get_loc(i) for i in df.index[df["signal"]==1]]
    sx = [df.index.get_loc(i) for i in df.index[df["signal"]==-1]]
    by = [float(df.iloc[i]["Low"].squeeze())*0.985  for i in bx]
```

```
sy = [float(df.iloc[i]["High"].squeeze())*1.015 for i in sx]
ms = 60 if mobile else 100
ax1.scatter(bx, by, marker="^", s=ms, color=C["buy"],  zorder=6, label="買い")
ax1.scatter(sx, sy, marker="v", s=ms, color=C["sell"], zorder=6, label="売り")

tc = {"上昇":"#3fb95015","下降":"#f8514915","横ばい":"none"}
prev, start = None, 0
for i,(_,row) in enumerate(df.iterrows()):
    t = row["trend"]
    if t!=prev:
        if prev and tc[prev]!="none": ax1.axvspan(start,i,color=tc[prev],lw=0
        start, prev = i, t
if prev and tc[prev]!="none": ax1.axvspan(start,len(df),color=tc[prev],lw=0,z

tnow = df["trend"].iloc[-1]
tclr = {"上昇":C["buy"],"下降":C["sell"],"横ばい":C["neutral"]}
ax1.text(0.995,0.97,f"トレンド:{tnow}", transform=ax1.transAxes,
         ha="right",va="top",fontsize=8 if mobile else 10,fontweight="bold",c
ax1.set_ylabel("株価", color=C["sub"], fontsize=6)
ax1.legend(loc="upper left", fontsize=5.5, framealpha=0.3,
           facecolor=C["bg"], edgecolor=C["grid"], labelcolor=C["text"])

ax2 = axes[1]
vc = [C["cup"] if cv[i]>=df["Open"].squeeze().values[i] else C["cdn"] for i i
ax2.bar(x, df["Volume"].squeeze().values, color=vc, alpha=0.7, width=0.8)
ax2.plot(x, df["Vol_MA20"].values, color=C["sma25"], lw=0.9)
ax2.set_ylabel("出来高", color=C["sub"], fontsize=6)
ax2.yaxis.set_major_formatter(
    plt.FuncFormatter(lambda v,_: f"{v/1e6:.0f}M" if v>=1e6 else f"{int(v/1e3

ax3 = axes[2]
hist = df["MACD_hist"].values
ax3.bar(x, hist, color=[C["hup"] if h>=0 else C["hdn"] for h in hist], alpha=
ax3.plot(x, df["MACD"].values,     color=C["macd"], lw=1.0, label="MACD")
ax3.plot(x, df["MACD_sig"].values, color=C["msig"], lw=0.9, ls="--", label="S
ax3.axhline(0, color=C["grid"], lw=0.7)
ax3.set_ylabel("MACD", color=C["sub"], fontsize=6)
ax3.legend(loc="upper left", fontsize=5.5, framealpha=0.2,
           facecolor=C["bg"], edgecolor="none", labelcolor=C["text"])

ax4 = axes[3]
ax4.plot(x, df["RSI"].values, color=C["rsi"], lw=1.1)
ax4.axhline(70, color=C["sell"], lw=0.8, ls="--")
ax4.axhline(50, color=C["grid"], lw=0.5)
ax4.axhline(30, color=C["buy"],  lw=0.8, ls="--")
ax4.fill_between(x, df["RSI"].values, 70, where=df["RSI"].values>=70, alpha=0
ax4.fill_between(x, df["RSI"].values, 30, where=df["RSI"].values<=30, alpha=0
```

```
    ax4.set_ylim(0,100); ax4.set_ylabel("RSI", color=C["sub"], fontsize=6)

    ax5 = axes[4]
    ax5.plot(x, df["Stoch_K"].values, color=C["macd"], lw=1.1, label="%K")
    ax5.plot(x, df["Stoch_D"].values, color=C["msig"], lw=0.9, ls="--", label="%D
    ax5.axhline(80, color=C["sell"], lw=0.8, ls="--")
    ax5.axhline(20, color=C["buy"],  lw=0.8, ls="--")
    ax5.fill_between(x, df["Stoch_K"].values, 80, where=df["Stoch_K"].values>=80,
    ax5.fill_between(x, df["Stoch_K"].values, 20, where=df["Stoch_K"].values<=20,
    ax5.set_ylim(0,100); ax5.set_ylabel("Stoch", color=C["sub"], fontsize=6)
    ax5.legend(loc="upper left", fontsize=5.5, framealpha=0.2,
               facecolor=C["bg"], edgecolor="none", labelcolor=C["text"])

    plt.close("all")
    return fig

# ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
#  Streamlit UI
# ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━

st.set_page_config(
    page_title="📈 トレンドシグナル",
    page_icon="📈",
    layout="wide",
    initial_sidebar_state="collapsed",
)

st.markdown("""
<style>
  .stApp,[data-testid="stAppViewContainer"]{background:#0d1117!important;color:#e
  section[data-testid="stSidebar"]{background:#161b22!important;}
  [data-testid="stHeader"]{background:#0d1117!important;}

  .badge-buy    {background:#1a3d24;color:#3fb950;border:1.5px solid #3fb950;
                 border-radius:10px;padding:7px 16px;font-size:1.05rem;font-weigh
  .badge-sell   {background:#3d1a1a;color:#f85149;border:1.5px solid #f85149;
                 border-radius:10px;padding:7px 16px;font-size:1.05rem;font-weigh
  .badge-neutral{background:#1a1f3d;color:#58a6ff;border:1.5px solid #58a6ff;
                 border-radius:10px;padding:7px 16px;font-size:1.05rem;font-weigh

  [data-testid="stMetricValue"]{color:#e6edf3!important;font-size:1.15rem!importa
  [data-testid="metric-container"]{background:#161b22;border-radius:10px;
                                   border:1px solid #21262d;padding:10px 14px;}

  .stButton>button{background:#21262d;color:#e6edf3;border:1px solid #30363d;
                   border-radius:8px;font-weight:600;}
  .stButton>button:hover{background:#388bfd22;border-color:#58a6ff;}
```

```
  .stTextInput>div>div>input{background:#161b22!important;color:#e6edf3!important
                              border:1px solid #30363d!important;border-radius:8p
                              font-size:1rem!important;}
  .stSelectbox>div>div{background:#161b22!important;}

  .stTabs [data-baseweb="tab"]{color:#8b949e;font-size:0.9rem;}
  .stTabs [aria-selected="true"]{color:#e6edf3!important;}
  .stDataFrame{border:1px solid #21262d!important;border-radius:8px;}

  .utime{color:#8b949e;font-size:0.72rem;}
  .log-row{border-bottom:1px solid #21262d;padding:3px 0;font-size:0.78rem;}

  @media(max-width:768px){
    .stApp{font-size:13px;}
    [data-testid="stMetricValue"]{font-size:0.95rem!important;}
    .badge-buy,.badge-sell,.badge-neutral{font-size:0.9rem;padding:5px 10px;}
    h1{font-size:1.3rem!important;}
    .block-container{padding:0.5rem 0.8rem!important;}
  }
  h1,h2,h3{color:#e6edf3!important;}
  hr{border-color:#21262d;}
</style>
""", unsafe_allow_html=True)

# — セッション初期化
defaults = {
    "result":None, "code":"", "period":"1y", "interval":"1d",
    "auto_on":False, "auto_interval":"15分", "update_log":[], "next_update":None,
}
for k,v in defaults.items():
    if k not in st.session_state: st.session_state[k] = v

# ━━━ サイドバー ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━

with st.sidebar:
    st.markdown("### ⚙️ 設定")
    period   = st.selectbox("取得期間", PERIODS,   index=3, format_func=lambda x: 
    interval = st.selectbox("時間軸",   INTERVALS, index=0, format_func=lambda x:

    st.divider()
    st.markdown("### ⏰ 自動更新")
    auto_int = st.selectbox("更新間隔", list(UPDATE_OPTS.keys()), index=2,
                            disabled=st.session_state["auto_on"])

    ac1, ac2 = st.columns(2)
    with ac1:
```

```
        if st.button("▶ 開始", use_container_width=True,
                     disabled=st.session_state["auto_on"] or not st.session_state
            st.session_state.update({
                "auto_on": True, "auto_interval": auto_int,
                "next_update": time.time() + UPDATE_OPTS[auto_int],
            })
            st.rerun()
    with ac2:
        if st.button("⏹ 停止", use_container_width=True,
                     disabled=not st.session_state["auto_on"]):
            st.session_state["auto_on"] = False
            st.rerun()

    if st.session_state["auto_on"]:
        st.success(f"● {st.session_state['auto_interval']}ごと自動更新中")
    else:
        st.caption("○ 停止中")

    if st.session_state["update_log"]:
        st.divider()
        st.markdown("### 📋 更新ログ")
        for e in st.session_state["update_log"][:8]:
            sig = e["シグナル"]
            col = "#3fb950" if "買" in sig else ("#f85149" if "売" in sig else "#5
            st.markdown(
                f"<div class='log-row'>"
                f"<span style='color:#8b949e'>{e['時刻']}</span> "
                f"<b>{e['銘柄']}</b> "
                f"<span style='color:{col}'>{sig}</span> {e['終値']}"
                f"</div>", unsafe_allow_html=True)

# ━━ メインエリア ━━━
```

```
# 「分析」実行
if fetch_btn and typed.strip():
    code = typed.strip().upper()
    fetch_and_analyze.clear()
    with st.spinner(f"📡 {code} を取得中..."):
        res = fetch_and_analyze(code, period, interval)
    if res and "df" in res:
        st.session_state.update({"result":res,"code":code,"period":period,"interv
        st.rerun()
    else:
        err = res.get("error","不明") if res else "データなし"
        st.error(f"取得失敗: {err}")

# 「今すぐ更新」
if refresh_btn and st.session_state["code"]:
    fetch_and_analyze.clear()
    with st.spinner("更新中..."):
        res = fetch_and_analyze(st.session_state["code"],
                                st.session_state["period"],
                                st.session_state["interval"])
    if res and "df" in res:
        st.session_state["result"] = res
        df_r = res["df"]
        log = st.session_state["update_log"]
        log.insert(0,{"時刻": res["fetched_at"][11:], "銘柄": st.session_state["co
                      "シグナル": sig_label(int(df_r["signal"].iloc[-1])),
                      "終値": f"{float(df_r['Close'].squeeze().iloc[-1]):,.1f}"})
        st.session_state["update_log"] = log[:30]
        st.rerun()

# — 自動更新タイミングチェック
if (st.session_state["auto_on"] and st.session_state["code"]
        and st.session_state["next_update"]
        and time.time() >= st.session_state["next_update"]):
    fetch_and_analyze.clear()
    with st.spinner("🔄 自動更新中..."):
        res = fetch_and_analyze(st.session_state["code"],
                                st.session_state["period"],
                                st.session_state["interval"])
    if res and "df" in res:
        st.session_state["result"] = res
        df_r = res["df"]
        log = st.session_state["update_log"]
        log.insert(0,{"時刻": res["fetched_at"][11:], "銘柄": st.session_state["co
                      "シグナル": sig_label(int(df_r["signal"].iloc[-1])),
                      "終値": f"{float(df_r['Close'].squeeze().iloc[-1]):,.1f}"})
        st.session_state["update_log"] = log[:30]
```

```
    st.session_state["next_update"] = time.time() + UPDATE_OPTS[st.session_state[

# — 人気銘柄クイックボタン（初期画面）
result = st.session_state.get("result")
if result is None:
    st.info("👆 銘柄コードを入力して「分析」ボタンを押すか、下のボタンから選んでください")
    st.divider()
    st.markdown("**📌 よく使われる銘柄**")
    cols = st.columns(5)
    for i,(c,n) in enumerate(POPULAR):
        if cols[i%5].button(f"{n}\n`{c}`", key=f"h_{c}", use_container_width=True
            fetch_and_analyze.clear()
            with st.spinner(f"{n} を取得中..."):
                res = fetch_and_analyze(c,"1y","1d")
            if res and "df" in res:
                st.session_state.update({"result":res,"code":c,"period":"1y","int
                st.rerun()
    st.stop()

# ━━━ 分析結果表示 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━

df   = result["df"]
code = st.session_state["code"]
per  = st.session_state["period"]
inv  = st.session_state["interval"]
lat  = df.iloc[-1]

close_v = float(df["Close"].squeeze().iloc[-1])
prev_v  = float(df["Close"].squeeze().iloc[-2])
chg_pct = (close_v - prev_v) / prev_v * 100
chg_abs = close_v - prev_v
sig     = int(lat["signal"])
trend   = lat["trend"]
fat     = result["fetched_at"]

# ヘッダー
st.markdown(
    f"### {code}  {PERIOD_LBL[per]}/{INTERVAL_LBL[inv]}"
    f"  <span class='utime'>更新: {fat}</span>",
    unsafe_allow_html=True)

# シグナルバッジ
sig_map = {1:("badge-buy","▲ 買いシグナル"), -1:("badge-sell","▼ 売りシグナル"), 0:("
css_cls, sig_txt = sig_map[sig]
tcol = {"上昇":"#3fb950","下降":"#f85149","横ばい":"#58a6ff"}[trend]

r1,r2,r3 = st.columns([2,2,3])
```

```
with r1:
    st.markdown(f"<div class='{css_cls}'>{sig_txt}</div>", unsafe_allow_html=True
with r2:
    st.markdown(f"<div style='font-size:0.95rem;margin-top:9px;color:{tcol};font-
                f"📊 {trend}トレンド</div>", unsafe_allow_html=True)
with r3:
    bsc=int(lat["buy_sc"]); ssc=int(lat["sell_sc"])
    st.progress(bsc/7, text=f"買いスコア {bsc}/7")
    st.progress(ssc/7, text=f"売りスコア {ssc}/7")

st.divider()

# 指標（4列 × 2行）
m1,m2,m3,m4 = st.columns(4)
m1.metric("終値",      f"{close_v:,.1f}",      f"{chg_pct:+.2f}%")
m2.metric("RSI",       f"{float(lat['RSI']):.1f}",
          "買過ぎ" if float(lat['RSI'])>70 else ("売過ぎ" if float(lat['RSI'])<30 e
m3.metric("ADX",       f"{float(lat['ADX']):.1f}",
          "強トレンド" if float(lat['ADX'])>25 else "レンジ")
m4.metric("MACD",      f"{float(lat['MACD']):.2f}")

m5,m6,m7,m8 = st.columns(4)
m5.metric("前日比",    f"{chg_abs:+,.1f}")
m6.metric("Stoch%K",   f"{float(lat['Stoch_K']):.1f}")
m7.metric("SMA25",     f"{float(lat['SMA25']):.1f}")
m8.metric("BB幅",      f"{float(lat['BB_width']):.4f}")

st.divider()

# タブ
tab1, tab2, tab3 = st.tabs(["📊 チャート", "📅 シグナル履歴", "📋 データ"])

with tab1:
    mobile_mode = st.toggle("📱 モバイル表示（縦長）", value=False)
    with st.spinner("描画中..."):
        title = f"{code}  {PERIOD_LBL[per]}/{INTERVAL_LBL[inv]}  {len(df)}本"
        fig = make_chart(df, title, mobile=mobile_mode)
        st.pyplot(fig, use_container_width=True)

with tab2:
    sig_df = df[df["signal"]!=0].copy()
    if not sig_df.empty:
        rows = []
        for idx_h, rh in sig_df.iterrows():
            cl_v = float(df.loc[idx_h,"Close"].squeeze())
            rows.append({
                "日付":      idx_h.strftime("%Y/%m/%d"),
```

```
                "シグナル":  sig_label(int(rh["signal"])),
                "終値":      f"{cl_v:,.1f}",
                "買Sc":      int(rh["buy_sc"]),
                "売Sc":      int(rh["sell_sc"]),
                "RSI":       round(float(rh["RSI"]),1),
                "ADX":       round(float(rh["ADX"]),1),
                "トレンド":  rh["trend"],
            })
        st.dataframe(pd.DataFrame(rows).iloc[::-1].reset_index(drop=True),
                     use_container_width=True, height=400)
    else:
        st.info("この期間ではシグナル発生なし。期間を延ばすか時間軸を変えてください。")

    if st.session_state["update_log"]:
        st.markdown("#### 自動更新ログ（全件）")
        st.dataframe(pd.DataFrame(st.session_state["update_log"]),
                     use_container_width=True)

with tab3:
    show_cols = ["Open","High","Low","Close","Volume",
                 "SMA25","SMA75","RSI","ADX","MACD","signal","trend"]
    df_show = df[[c for c in show_cols if c in df.columns]].tail(60).copy()
    df_show.columns = [c if isinstance(c,str) else c[0] for c in df_show.columns]
    st.dataframe(df_show.style.format(precision=2), use_container_width=True, hei

# — 自動更新ループ（次回まで待機 → rerun）
if st.session_state["auto_on"] and st.session_state["next_update"]:
    remaining = int(st.session_state["next_update"] - time.time())
    if remaining > 0:
        st.caption(f"⏱ 次の自動更新まで約 {remaining}秒")
        time.sleep(min(remaining, 30))
        st.rerun()
    else:
        st.rerun()
```